

Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа программной инженерии

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине «Базы данных»

Вариант №1

Выполнил
студент гр. 5130904/30107



Голиков. В. С.

Руководитель

Юркин В. А.

Оглавление

Постановка задачи.....	3
Выполнение поставленной задачи	5
Заключение	16

Постановка задачи

Представления

1. Создать представление, отображающее всех водителей
2. Создать представление, отображающее все маршруты и количество автомобилей находящихся на каждом маршруте

Хранимые процедуры

- Без параметров
 1. Создать хранимую процедуру, выводящую все маршруты и среднее время проезда по ним в минутах
- С входными параметрами
 1. Создать хранимую процедуру, имеющую два параметра «автомобиль1» и «автомобиль2». Она должна возвращать маршруты, по которым «автомобиль1» проехал быстрее «автомобилья2». Если на каком-либо маршруте один из автомобилей не проезжал, такой маршрут не рассматривается.
- С выходными параметрами
 1. Создать хранимую процедуру с входным параметром «маршрут» и двумя выходными параметрами, возвращающими самое короткое время проезда по заданному маршруту и автомобиль, поставивший рекорд

Триггера

- Триггера на вставку
 1. Создать триггер, который не позволяет отправить в рейс автомобиль, который еще не вернулся в парк
- Триггера на модификацию
 1. Создать триггер, который при изменении маршрута (на который уже существуют ссылки из журнала) будет создавать новый маршрут, хранящий неизменный (прежний) вариант маршрута и подменять все ссылки на него. Таким образом, старые записи журнала будут по-прежнему ссылаться на маршрут, по которому они были созданы.
- Триггера на удаление
 1. Создать триггер, который при удалении водителя в случае наличия на него ссылок откатывает транзакцию

Курсоры

- Хранимая процедура для расчета премий

Необходимо создать хранимую процедуру, распределяющую премии в автопарке. Премияльные денежные средства выдаются трем водителям, проехавшим за минимальное время наибольшее количество маршрутов, причем 50% от премиальной суммы выдается первому водителю, а остальным двум выдается 30% и 20% в соответствии с их успехами.

Хранимая процедура должна иметь три входных параметра: дату начала, дату окончания и сумму премиальных, подлежащих распределению за требуемый период времени. Результатом работы хранимой процедуры должны быть три водителя с суммами их премиальных.

Предлагаемый алгоритм следующий: создаем 3 переменные для хранения идентификаторов водителей и 3 переменные для хранения их результатов. Создаем курсор, который пробегает по таблице персонала и, если автомобили данного водителя поставили рекорд в большем числе маршрутов, нежели уже выбранные водители, хранящиеся в переменных, то данный водитель вытесняет ранее сохраненного водителя с худшими результатами. Результатом работы хранимой процедуры будет выборка значений переменных с суммами, рассчитанными по описанному выше алгоритму.

Выполнение поставленной задачи

Представления

1.

```
Copy Copy to Query Editor

CREATE VIEW drivers AS
SELECT * FROM auto_personal
```

Messages

Query returned successfully in 76 msec.

2.

Query Query History

```
1 CREATE VIEW on_route AS
2 SELECT
3     routes.id,
4     routes.name,
5     COUNT(DISTINCT journal.auto_id)
6 FROM routes
7 LEFT JOIN journal ON routes.id = journal.route_id
8 AND journal.time_in IS NULL
9 GROUP BY routes.id, routes.name;
10
11 SELECT * FROM on_route
```

Data Output Messages Notifications

	id integer	name character varying (20)	count bigint
1	4	Северный путь	0
2	5	Южный путь	0
3	6	СПб-Москва	1
4	7	Мурманск-СПб	0

Хранимые процедуры

1. Текст процедуры:

```
CREATE OR REPLACE PROCEDURE get_routes_avg_time()
LANGUAGE plpgsql
AS $$
DECLARE
    route_record RECORD;
BEGIN
    RAISE NOTICE '% | % | %', ' ID ', ' Маршрут ', ' Ср. время (мин) ';
    RAISE NOTICE '-----|-----|-----';

    FOR route_record IN
        SELECT
            r.id,
            r.name,
            ROUND(AVG(EXTRACT(EPOCH FROM (j.time_in -
j.time_out))/60)::NUMERIC, 2) AS avg_minutes
        FROM routes r
        JOIN journal j ON r.id = j.route_id
        WHERE j.time_in IS NOT NULL
            AND j.time_out IS NOT NULL
        GROUP BY r.id, r.name
        ORDER BY avg_minutes
    LOOP
        RAISE NOTICE '%', FORMAT(' %-3s | %-12s | %-10s',
            route_record.id,
            route_record.name,
            route_record.avg_minutes);
    END LOOP;
END;
$$;
```

CALL get_routes_avg_time();

Data Output **Messages** Notifications

	ID	Маршрут	Ср. время (мин)
ЗАМЕЧАНИЕ:	5	Южный путь	0.32
ЗАМЕЧАНИЕ:	7	Мурманск-СПб	20.36
ЗАМЕЧАНИЕ:	6	СПб-Москва	5020.59

CALL

2. Текст процедуры:

```
CREATE OR REPLACE PROCEDURE compare_cars_time(
    car1_id INTEGER,
    car2_id INTEGER
)
LANGUAGE plpgsql
AS $$
DECLARE
    compare_record RECORD;
    car1_num TEXT;
    car2_num TEXT;
    routes_count INTEGER := 0;
BEGIN
    -- Получаем номера автомобилей для красивого вывода
    SELECT num INTO car1_num FROM auto WHERE id = car1_id;
    SELECT num INTO car2_num FROM auto WHERE id = car2_id;

    -- Заголовок
    RAISE NOTICE 'Авто 1 (ID:%): %', car1_id, COALESCE(car1_num, 'не найден');
    RAISE NOTICE 'Авто 2 (ID:%): %', car2_id, COALESCE(car2_num, 'не найден');
    RAISE NOTICE '%', FORMAT('%-6s | %-25s | %-15s | %-15s', 'ID',
'Mаршрут', 'Авто1 время', 'Авто2 время');

    -- Обрабатываем результаты сравнения
    FOR compare_record IN
        SELECT
            r.id,
            r.name,
            MIN(CASE WHEN j1.auto_id = car1_id THEN j1.time_in - j1.time_out
END) AS car1_time,
            MIN(CASE WHEN j2.auto_id = car2_id THEN j2.time_in - j2.time_out
END) AS car2_time
        FROM routes r
        JOIN journal j1 ON r.id = j1.route_id AND j1.auto_id = car1_id AND
j1.time_in IS NOT NULL
        JOIN journal j2 ON r.id = j2.route_id AND j2.auto_id = car2_id AND
j2.time_in IS NOT NULL
        GROUP BY r.id, r.name
        HAVING MIN(CASE WHEN j1.auto_id = car1_id THEN j1.time_in -
j1.time_out END) <
            MIN(CASE WHEN j2.auto_id = car2_id THEN j2.time_in - j2.time_out
END)
```

```

LOOP
  -- Вывод данных о маршруте
  RAISE NOTICE '%', FORMAT('%-6s | %-25s | %-15s | %-15s',
    compare_record.id,
    compare_record.name,
    date_trunc('second', compare_record.car1_time),
    date_trunc('second', compare_record.car2_time));

    routes_count := routes_count + 1;
END LOOP;
END;
$$;

CALL compare_cars_time(2,4);

```

Data Output Messages Notifications

ЗАМЕЧАНИЕ:	Авто 1 (ID:2): A777УФ_42		
ЗАМЕЧАНИЕ:	Авто 2 (ID:4): p789уд_98		
ЗАМЕЧАНИЕ:	ID	Маршрут	Авто1 время Авто2 время
ЗАМЕЧАНИЕ:	6	СПб-Москва	00:05:47 6 days 23:15:23
CALL			

3.

```
CREATE OR REPLACE PROCEDURE get_route_record(  
    IN route_name TEXT,  
    OUT min_time INTERVAL,  
    OUT record_car_num TEXT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    SELECT  
        MIN(j.time_in - j.time_out),  
        a.num  
    INTO  
        min_time,  
        record_car_num  
    FROM journal j  
    JOIN routes r ON j.route_id = r.id  
    JOIN auto a ON j.auto_id = a.id  
    WHERE r.name = route_name  
        AND j.time_in IS NOT NULL  
        AND j.time_out IS NOT NULL  
        AND j.time_in > j.time_out  
    GROUP BY r.id, a.id, a.num  
    ORDER BY MIN(j.time_in - j.time_out)  
    LIMIT 1;  
  
    IF min_time IS NULL THEN  
        min_time := '0':INTERVAL;  
        record_car_num := 'Нет данных';  
    END IF;  
END;  
$$;  
  
DO $$  
DECLARE  
    min_t INTERVAL;  
    car_num TEXT;  
BEGIN  
    CALL get_route_record('СПб-Москва', min_t, car_num);  
    RAISE NOTICE 'Рекорд: %, Автомобиль: %', date_trunc('second', min_t),  
car_num;  
END;  
$$;
```

ЗАМЕЧАНИЕ: Рекорд: 00:05:47, Автомобиль: A777УФ_42
D0

Триггера

1. Текст программы:

```
CREATE OR REPLACE FUNCTION check_car_available()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM journal
        WHERE auto_id = NEW.auto_id
        AND time_in IS NULL
        AND (TG_OP = 'INSERT' OR id != COALESCE(NEW.id, -1))
    ) THEN
        RAISE EXCEPTION 'Car with ID % is still on route and cannot be
dispatched again', NEW.auto_id;
    END IF;

    RETURN NEW;
END;
$$;
```

```
CREATE OR REPLACE TRIGGER prevent_double_dispatch
BEFORE INSERT OR UPDATE ON journal
FOR EACH ROW
EXECUTE FUNCTION check_car_available();
```

Query	Query History
1	INSERT INTO journal (auto_id, route_id, time_out)
2	VALUES (2, 5, NOW())

Data Output	Messages	Notifications
ERROR: Car with ID 2 is still on route and cannot be dispatched again CONTEXT: функция PL/pgSQL check_car_available(), строка 10, оператор RAISE		
ОШИБКА: Car with ID 2 is still on route and cannot be dispatched again SQL state: P0001		

2. Текст программы

```
CREATE OR REPLACE FUNCTION protect_route_history()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
DECLARE
    new_route_id INTEGER;
BEGIN
    IF EXISTS (SELECT 1 FROM journal WHERE route_id = OLD.id) THEN
        INSERT INTO routes (name)
        VALUES (OLD.name)
        RETURNING id INTO new_route_id;

        UPDATE journal
        SET route_id = new_route_id
        WHERE route_id = OLD.id;
    END IF;
    RAISE NOTICE 'route % was changed', OLD.id;
    RETURN NEW;
END;
$$;
```

```
CREATE TRIGGER protect_route_history_trigger
BEFORE UPDATE ON routes
FOR EACH ROW
EXECUTE FUNCTION protect_route_history();
```

Query Query History

```
1  INSERT INTO journal (auto_id, route_id, time_out, time_in) VALUES
2  (3, 13, '2024-01-15 08:00', '2024-01-15 09:00');
3
4  UPDATE routes SET name = 'new test way' WHERE id = 13;
```

Data Output Messages Notifications

```
ЗАМЕЧАНИЕ: route 13 was changed
UPDATE 1
```

3. Текст программы

```
CREATE OR REPLACE FUNCTION prevent_personal_delete()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF EXISTS (
        SELECT 1
        FROM auto
        WHERE personal_id = OLD.id
    ) THEN
        RAISE EXCEPTION 'Can not delete driver ID %', OLD.id;
    END IF;

    RETURN OLD;
END;
$$
```

```
1 DELETE FROM auto_personal WHERE id = 5
```

Data Output Messages Notifications

ERROR: Can not delete driver ID 5

CONTEXT: функция PL/pgSQL prevent_personal_delete(), строка 8, оператор RAISE

Kypcop

```
CREATE OR REPLACE FUNCTION calculate_bonuses(
    start_date DATE,
    end_date DATE,
    total_bonus_amount DECIMAL
)
RETURNS TABLE(
    driver_id INTEGER,
    driver_name TEXT,
    record_routes_count INTEGER,
    bonus_amount DECIMAL
)
LANGUAGE plpgsql
AS $$
DECLARE
    driver_rec RECORD;
    top1_id INTEGER := NULL;
    top2_id INTEGER := NULL;
    top3_id INTEGER := NULL;
    top1_count INTEGER := 0;
    top2_count INTEGER := 0;
    top3_count INTEGER := 0;
    current_count INTEGER;
BEGIN
    FOR driver_rec IN (
        SELECT
            ap.id as driver_id,
            ap.first_name || ' ' || ap.last_name || ' ' || ap.father_name as driver_name,
            COUNT(DISTINCT r.id) as record_routes_count
        FROM auto_personal ap
        JOIN auto a ON ap.id = a.personal_id
        JOIN journal j ON a.id = j.auto_id
        JOIN routes r ON j.route_id = r.id
        WHERE j.time_out BETWEEN start_date AND end_date
            AND j.time_in IS NOT NULL
            AND j.time_in > j.time_out
        --проверка на рекорд
        AND (j.auto_id, j.route_id, (j.time_in - j.time_out)) IN (
            SELECT j2.auto_id, j2.route_id, MIN(j2.time_in - j2.time_out)
            FROM journal j2
            WHERE j2.time_out BETWEEN start_date AND end_date
                AND j2.time_in IS NOT NULL
                AND j2.time_in > j2.time_out
                AND j2.route_id = j.route_id
```

```

        GROUP BY j2.route_id, j2.auto_id
    )
    GROUP BY ap.id, ap.first_name, ap.last_name, ap.father_name
    ORDER BY record_routes_count DESC
)
LOOP
    current_count := driver_rec.record_routes_count;

    IF current_count > top1_count THEN
        top3_id := top2_id;
        top3_count := top2_count;
        top2_id := top1_id;
        top2_count := top1_count;
        top1_id := driver_rec.driver_id;
        top1_count := current_count;
    ELSIF current_count > top2_count THEN
        top3_id := top2_id;
        top3_count := top2_count;
        top2_id := driver_rec.driver_id;
        top2_count := current_count;
    ELSIF current_count > top3_count THEN
        top3_id := driver_rec.driver_id;
        top3_count := current_count;
    END IF;
END LOOP;

IF top1_id IS NOT NULL THEN
    driver_id := top1_id;
    SELECT first_name || ' ' || last_name || ' ' || father_name
    INTO driver_name
    FROM auto_personal
    WHERE id = top1_id;
    record_routes_count := top1_count;
    bonus_amount := total_bonus_amount * 0.5;
    RETURN NEXT;
END IF;

IF top2_id IS NOT NULL THEN
    driver_id := top2_id;
    SELECT first_name || ' ' || last_name || ' ' || father_name
    INTO driver_name
    FROM auto_personal
    WHERE id = top2_id;
    record_routes_count := top2_count;

```

```

        bonus_amount := total_bonus_amount * 0.3;
        RETURN NEXT;
    END IF;

    IF top3_id IS NOT NULL THEN
        driver_id := top3_id;
        SELECT first_name || ' ' || last_name || ' ' || father_name
        INTO driver_name
        FROM auto_personal
        WHERE id = top3_id;
        record_routes_count := top3_count;
        bonus_amount := total_bonus_amount * 0.2;
        RETURN NEXT;
    END IF;
END;
$$

```

Query

Query History

1

SELECT * FROM calculate_bonuses('2024-01-01', '2027-01-31', 10000);

Data Output

Messages

Notifications

≡+

📄

▼

📋

▼

🗑

🗄

⬇

📈

SQL

Showing rows: 1 to 3

✎

Page 1

	driver_id integer	driver_name text	record_routes_count integer	bonus_amount numeric
1	11	Мария Сидорова Ивано...	3	5000.0
2	10	Иван Петров Сергеевич	2	3000.0
3	12	Алексей Иванов Петров...	2	2000.0

Заключение

В данной лабораторной работе мною были изучены такие элементы баз данных, как представления, хранимые процедуры, триггеры и курсоры. В процессе выполнения работы были созданы конечные процедуры и триггеры, которые в дальнейшем будут использованы при написании курсовой работы.